KELOMPOK 3 · PEMROGRAMAN BERORIENTASI OBJEK

Naskah Presentasi

Sistem Manajemen Data Pasien Rumah Sakit

Naskah lisan untuk 37 slide yang tampil di PPT. Teksnya sama persis dengan catatan pembicara di ketiga file PPT (Navy Klasik, Swiss Modern, dan Medis Hijau).

No	Pembicara	Slide	Kata	Waktu baca	Waktu tampil
1	**Riswan**	1–5	267	± 2 menit 15 detik	± 2 menit 45 detik
2	**Yus Aidil**	6–8	298	± 2 menit 30 detik	± 3 menit 5 detik
3	**Fahmi**	9–12	336	± 2 menit 50 detik	± 3 menit 30 detik
4	**Diva Nurislamia Agfa**	13–16	320	± 2 menit 40 detik	± 3 menit 20 detik
5	**Rosa Ramadani**	17–19	276	± 2 menit 20 detik	± 2 menit 50 detik
6	**Sofiang**	20–22	260	± 2 menit 10 detik	± 2 menit 40 detik
7	**Revalinda**	23–26	323	± 2 menit 40 detik	± 3 menit 20 detik
8	**Andi Rezky Nabila**	27–30	330	± 2 menit 45 detik	± 3 menit 25 detik
9	**Alfira Yunita Syam**	31–37	394	± 3 menit 15 detik	± 4 menit 5 detik
	Total	1–37	**2804**	**± 23 menit 20 detik**	**± 29 menit 10 detik**

Waktu baca dihitung 120 kata per menit. Waktu tampil ditambah ±25% untuk jeda, klik, dan menunjuk kode. Durasi di PPT adalah waktu tampil yang dibulatkan per anggota; totalnya ± 29,5 menit.

Cara membaca naskah

- KLIK → = ganti ke slide berikutnya.
- *(tunjuk: baris 14)* = arahkan pointer atau kursor ke bagian yang disebut.
- **Huruf tebal** = kata kunci, ucapkan lebih jelas dan pelan.
- *(jeda)* = berhenti sejenak, biasanya sebelum kalimat estafet ke pembicara berikutnya.
- Tabel **Penjelasan kode** di bawah naskah menjelaskan fungsi setiap baris kode yang tampil di slide. Pelajari sebelum maju, supaya bisa menjelaskan kode dan menjawab pertanyaan.
- Semua output di slide dan naskah berasal dari program yang **benar-benar dijalankan**.

Tips

- Pahami alurnya, jangan menghafal kata per kata. Pakai **kata kunci** di tiap bagian sebagai pegangan.
- Tekan **Alt+F5** (Presenter View) agar naskah ini terlihat di layar laptop, tetapi tidak di proyektor.
- Bacakan angka dengan format Indonesia, misalnya `8,367.48` dibaca “delapan ribu tiga ratus enam puluh tujuh koma empat delapan”.
- Latihan minimal dua kali penuh dengan stopwatch, termasuk pergantian pembicara.

BAGIAN 1 | RISWAN

Pembukaan & Gambaran Umum Project

Slide 1–5 • **Durasi** ± 3 menit • **Waktu baca** ± 2 menit 15 detik • **Waktu tampil** ± 2 menit 45 detik • 267 kata

Kata kunci: Sistem Manajemen Data Pasien Rumah Sakit · Situasinya · Masalahnya · pertanyaannya · Jawaban · data nyata 984 pasien · menu interaktif · python main.py

SLIDE 1 Sistem Manajemen Data Pasien Rumah Sakit

± 15 detik

Selamat pagi Bapak/Ibu dosen dan teman-teman semua. Kami dari **Kelompok 3** akan mempresentasikan project berjudul **Sistem Manajemen Data Pasien Rumah Sakit**, yang kami buat dengan bahasa Python untuk menerapkan konsep pemrograman berorientasi objek. KLIK →

SLIDE 2 Pembagian Materi Kelompok 3

± 25 detik

Presentasi kami dibagi menjadi sembilan bagian, sesuai jumlah anggota. Saya, Riswan, membuka presentasi. Lalu Yus Aidil menjelaskan dataset, Fahmi tentang class dan object, Diva tentang enkapsulasi, Rosa dan Sofiang tentang inheritance, Revalinda tentang komposisi dan validasi data medis, Andi tentang program utama dan menu interaktif, dan Alfira menunjukkan hasil, demo menu, serta kesimpulan. KLIK →

SLIDE 3 Riswan

± 5 detik

Kita mulai dari bagian pertama, yaitu gambaran umum project. KLIK →

SLIDE 4 OOP Membuat Data Pasien Lebih Rapi dan Aman

± 40 detik

(tunjuk: kartu S) **Situasinya**, rumah sakit mencatat banyak data pasien setiap hari, dari identitas sampai biaya. *(tunjuk: kartu C)* **Masalahnya**, data seperti ini mudah berantakan dan rawan salah input, misalnya umur negatif atau kepuasan tujuh, padahal skalanya hanya satu sampai lima. *(tunjuk: kartu Q)* Jadi **pertanyaannya**, bagaimana mengelola data ini secara terstruktur dan aman? *(tunjuk: kartu A)* **Jawaban** kami adalah program Python berbasis OOP: setiap pasien menjadi object yang datanya dijaga. Kami memakai **data nyata 984 pasien** dan menerapkan tiga konsep: class dan object, enkapsulasi, serta inheritance. KLIK →

SLIDE 5 Lima File, Satu Alur: dari CSV Menjadi Ringkasan

± 50 detik

(tunjuk: alur program) Alurnya: program membaca file CSV dan mengubah setiap baris menjadi object; baris yang datanya salah dilewati dan nomornya dicatat. Object disimpan di list, lalu program menampilkan lima rekam medis pertama, ringkasan, dan demo validasi. Terakhir ada **menu interaktif** untuk melihat, mencari, dan menambah pasien. *(tunjuk: kartu file)* Kodenya dibagi menjadi lima file, satu class satu file, ditambah `main.py` sebagai program

utama. Dengan pembagian ini, setiap file punya satu tugas yang jelas dan mudah dicari. *(tunjuk: kotak hitam)* Perintah **python main.py** fungsinya menjalankan program utama. Program hanya memakai library bawaan, jadi tidak perlu instalasi tambahan. *(jeda)* Selanjutnya, **Yus Aidil** akan menjelaskan dataset yang kami gunakan. **KLIK →**

Kalimat penutup bagianmu: "Selanjutnya, Yus Aidil akan menjelaskan dataset yang kami gunakan."

Persiapan tanya jawab

T **Kenapa memakai OOP, bukan program biasa (prosedural)?**

J Karena data pasien punya ciri (atribut) dan perilaku (method) yang jelas. Dengan OOP, setiap pasien menjadi object yang rapi, datanya terlindungi lewat enkapsulasi, dan kodenya bisa dipakai ulang lewat inheritance.

T **Kenapa satu class satu file?**

J Supaya kode mudah dicari, dibaca, dan dikembangkan. Setiap file hanya punya satu tanggung jawab.

T **Kenapa hanya memakai library bawaan?**

J Supaya program bisa langsung dijalankan di komputer mana pun yang sudah terpasang Python, tanpa instalasi tambahan.

BAGIAN 2 | YUS AIDIL

Dataset & Aturan Data

Slide 6–8 • **Durasi** ± 3 menit • **Waktu baca** ± 2 menit 30 detik • **Waktu tampil** ± 3 menit 5 detik • 298 kata

Kata kunci: hospital data analysis.csv · 984 baris · 10 kolom · tidak ada satu pun nilai kosong · 264 pasien dirawat ulang · tanpa menambah atribut lain · Readmission · jenis class

SLIDE 6 Yus Aidil

± 10 detik

Terima kasih, Riswan. Saya Yus Aidil, akan menjelaskan dataset dan aturan data yang kami pakai. KLIK →

SLIDE 7 984 Data Pasien Nyata, Bersih Tanpa Nilai Kosong

± 1 menit 10 detik

Data kami berasal dari file **hospital data analysis.csv**. *(tunjuk: empat angka)* Isinya **984 baris** data pasien dengan **10 kolom**, dan setelah kami periksa, **tidak ada satu pun nilai kosong**. Jenis kondisi penyakitnya ada 15. *(tunjuk: kotak CSV)* Baris pertama file disebut header, berisi nama kolom: ID pasien, umur, jenis kelamin, kondisi, prosedur, biaya, lama rawat, status rawat ulang, hasil perawatan, dan kepuasan. Baris kedua dan seterusnya adalah data pasien, dipisahkan tanda koma; misalnya pasien nomor satu berumur empat puluh lima tahun dengan penyakit jantung. Umur pasien di data ini antara 25 sampai 78 tahun, dan lama rawatnya antara 1 sampai 76 hari. *(tunjuk: grafik sebaran)* Sebarannya: 524 pasien perempuan dan 460 laki-laki. Sebanyak **264 pasien dirawat ulang**, sedangkan 720 tidak. Untuk hasil perawatan, 591 pasien Recovered atau sembuh, dan 393 berstatus Stable. Kebersihan data ini penting, karena program kami memeriksa setiap nilai; baris yang melanggar aturan akan dilewati. KLIK →

SLIDE 8 Setiap Kolom Menjadi Atribut, Tanpa Atribut Tambahan

± 1 menit 10 detik

Setiap kolom kami jadikan atribut di class, **tanpa menambah atribut lain**. *(tunjuk: tabel)* Misalnya kolom Age menjadi atribut umur di class Orang, Satisfaction menjadi kepuasan di class Pasien, dan Cost menjadi biaya di class RekamMedis. Jadi data identitas ada di Orang dan Pasien, sedangkan data perawatan ada di RekamMedis, sehingga struktur class mengikuti struktur data aslinya. Di kolom paling kanan ada aturan nilainya: jenis kelamin hanya Male atau Female, hasil hanya Recovered atau Stable, kepuasan satu sampai lima, umur dan biaya harus positif, lama rawat bilangan bulat lebih dari nol, serta kondisi dan prosedur tidak boleh kosong. Aturan ini nanti dijaga oleh setter. Pemetaan ini juga memudahkan saat membaca CSV, karena setiap kolom langsung jelas masuk ke atribut mana. *(tunjuk: baris kuning)* Khusus kolom **Readmission**, nilainya tidak disimpan sebagai

atribut, tetapi diwakili oleh **jenis class**: Yes menjadi PasienRawatUlang, No menjadi Pasien.

(jeda) Selanjutnya, **Fahmi** akan menjelaskan konsep class dan object. KLIK →

Kalimat penutup bagianmu: “Selanjutnya, Fahmi akan menjelaskan konsep class dan object.”

Persiapan tanya jawab

T **Data ini data nyata atau buatan sendiri?**

J Data nyata dari file CSV, bukan data karangan. Program membaca file tersebut secara langsung setiap kali dijalankan.

T **Bagaimana kalau ada data yang tidak sesuai aturan?**

J Baris itu dilewati dengan peringatan, nomor barisnya dicatat, dan jumlah baris yang dilewati ditampilkan setelah data dimuat. Program tetap berjalan. Pada data asli, 0 baris dilewati.

T **Kenapa Readmission tidak dijadikan atribut?**

J Karena pasien rawat ulang adalah jenis khusus dari pasien. Jadi cukup diwakili oleh class PasienRawatUlang, tanpa atribut tambahan.

BAGIAN 3 | FAHMI

Konsep Class & Object (class Orang)

Slide 9–12 • **Durasi** ± 3,5 menit • **Waktu baca** ± 2 menit 50 detik • **Waktu tampil** ± 3 menit 30 detik • 336 kata

> **Kata kunci:** cetakan · hasil cetakannya · 720 object Pasien · Orang · constructor · self · tampilkan_info · PasienRawatUlang · Pasien · daftar_rekam · 1968 object

SLIDE 9 Fahmi

± 5 detik

Terima kasih, Yus. Saya Fahmi, akan menjelaskan konsep class dan object. **KLIK →**

SLIDE 10 Class adalah Cetakan, Object adalah Hasilnya

± 45 detik

Class bisa diibaratkan **cetakan** atau blueprint, sedangkan object adalah **hasil cetakannya**. Analoginya seperti formulir pendaftaran pasien: bentuk formulirnya sama untuk semua orang, tetapi isinya berbeda untuk setiap pasien. *(tunjuk: kotak class Pasien)* Ini class Pasien sebagai cetakan. *(tunjuk: tiga kartu object)* Dari satu cetakan, kita bisa membuat banyak object. Baris pertama data menjadi object pasien ID 1, umur 45 tahun, perempuan, dengan kepuasan 4. Baris ketiga menjadi pasien ID 3, umur 32 tahun. Baris keenam menjadi pasien ID 6, laki-laki berumur 68 tahun. Dari seluruh data, terbentuk **720 object Pasien**. Di project kami ada empat class: Orang, Pasien, PasienRawatUlang, dan RekamMedis. **KLIK →**

SLIDE 11 Class Orang Menjadi Dasar Semua Pasien

± 1 menit

Ini kode class **Orang** di file `orang.py`. *(tunjuk: baris 6)* Baris 6, `class Orang`, fungsinya membuat class baru bernama Orang. Baris 7 adalah docstring, yaitu keterangan singkat isi class. *(tunjuk: baris 9)* Baris 9, `def __init__`, adalah **constructor**: method yang berjalan otomatis setiap kali object dibuat, dan menerima umur serta jenis kelamin. Kata **self** menunjuk ke object itu sendiri, sehingga setiap object menyimpan datanya masing-masing. Baris 10 dan 11 hanya komentar, tidak dijalankan. *(tunjuk: baris 12 dan 13)* Baris 12 dan 13 mengisi umur dan jenis kelamin ke object. Nilainya dikirim lewat setter, jadi langsung divalidasi; bagian ini nanti dijelaskan Diva. *(tunjuk: baris 39 sampai 42)* Lalu baris 39 membuat method **tampilkan_info**, dan baris 41 serta 42 mencetak umur dan jenis kelamin ke layar. Method ini nanti ditulis ulang, atau di-override, oleh class turunannya. **KLIK →**

Penjelasan kode · orang.py baris 6–13

Baris	Kode	Fungsinya
6	`class Orang:`	Membuat class baru bernama Orang, yaitu cetakan data dasar seseorang.
7	`"""Menyimpan data dasar seseorang: umur dan jenis kelamin."""`	Docstring: keterangan singkat isi class (tidak dijalankan sebagai perintah).

Baris	Kode	Fungsinya
9	`def __init__(self, umur, jenis_kelamin):`	Constructor `__init__`: berjalan otomatis saat object dibuat dan menerima umur serta jenis kelamin. `self` adalah object itu sendiri.
10	`# [ENKAPSULASI] Nilai diisi lewat setter, jadi langsung divalidasi`	Komentar (tidak dijalankan): nilai diisi lewat setter supaya langsung divalidasi.
11	`# sebelum disimpan ke atribut private (self.__umur, self.__jenis_kelamin).`	Komentar lanjutan: nilai yang lolos disimpan di atribut private.
12	`self.umur = umur`	Mengisi umur lewat setter `umur`, jadi nilainya diperiksa dulu sebelum disimpan.
13	`self.jenis_kelamin = jenis_kelamin`	Mengisi jenis kelamin lewat setter `jenis_kelamin`.

Penjelasan kode · orang.py baris 38–42

Baris	Kode	Fungsinya
38	`# ---------- Method ----------`	Komentar penanda bagian method.
39	`def tampilkan_info(self):`	Method `tampilkan_info()` untuk menampilkan data dasar; nanti di-override class turunan.
40	`"""Menampilkan info dasar. Method ini di-override oleh class turunan."""`	Docstring: keterangan method.
41	`print(f"Umur : {self.umur} tahun")`	Mencetak umur (dibaca lewat property `self.umur`).
42	`print(f"Jenis Kelamin : {self.jenis_kelamin}")`	Mencetak jenis kelamin.

SLIDE 12 984 Baris CSV Menjadi 1968 Object

± 55 detik

Lalu bagaimana object dibuat dari file CSV? Ini potongan fungsi `buat_pasien` di `main.py`. *(tunjuk: baris 45 dan 46)* Baris 45 memeriksa kolom Readmission. Kalau nilainya Yes, baris 46 membuat dan mengembalikan object **PasienRawatUlang**. *(tunjuk: baris 47 dan 48)* Kalau nilainya No, baris 48 membuat object **Pasien**. Jadi pemilihan class terjadi otomatis, dan bagian lain program tidak perlu memeriksa jenis pasien secara manual. *(tunjuk: baris 73 sampai 76)* Potongan kedua dari fungsi `muat_data`: baris 73 menyiapkan pembaca CSV, baris 74 mengulang setiap baris data, dan baris 76 membuat object RekamMedis lalu menambahkannya ke list **daftar_rekam** dengan append. *(tunjuk: angka 1968)* Karena ada 984 baris dan setiap baris menghasilkan dua object, totalnya **1968 object**: 720 Pasien, 264 PasienRawatUlang, dan 984 RekamMedis. *(jeda)* Selanjutnya, **Diva** akan menjelaskan bagaimana data di dalam object ini dilindungi dengan enkapsulasi. **KLIK →**

Penjelasan kode · main.py baris 44–48

Baris	Kode	Fungsinya
44	`# [INHERITANCE] Kolom Readmission menentukan class object yang dibuat`	Komentar: kolom Readmission menentukan class object yang dibuat.
45	`if readmission == "Yes":`	Jika Readmission bernilai "Yes" …
46	`return PasienRawatUlang(patient_id, umur, jenis_kelamin, kepuasan)`	… buat dan kembalikan object `PasienRawatUlang`.
47	`elif readmission == "No":`	Jika bernilai "No" …
48	`return Pasien(patient_id, umur, jenis_kelamin, kepuasan)`	… buat dan kembalikan object `Pasien`.

Penjelasan kode · main.py baris 73–76

Baris	Kode	Fungsinya
73	`pembaca = csv.DictReader(file, restval="")`	`csv.DictReader` membaca setiap baris sebagai dictionary berdasarkan nama kolom, misalnya `baris["Age"]`. `restval=""` mengisi kolom yang hilang dengan teks kosong, sehingga baris yang kolomnya kurang ditolak validasi, bukan menghentikan program.
74	`for baris in pembaca:`	Mengulang setiap baris data di file.
75	`try:`	`try`: mencoba membuat object dari baris ini.
76	`daftar_rekam.append(buat_rekam_medis(baris))`	Membuat object rekam medis lalu menambahkannya ke list dengan append.

> **Kalimat penutup bagianmu:** “Selanjutnya, Diva akan menjelaskan bagaimana data di dalam object ini dilindungi dengan enkapsulasi.”

Persiapan tanya jawab

T **Apa perbedaan class dan object?**

J Class adalah rancangan (cetakan), sedangkan object adalah hasil nyatanya. Satu class bisa menghasilkan banyak object, contohnya class Pasien yang menghasilkan ratusan object pasien.

T **Apa fungsi self?**

J self menunjuk ke object yang sedang dipakai, sehingga setiap object menyimpan datanya masing-masing.

T **Berapa object yang dibuat program?**

J 1968 object: 984 object pasien (720 Pasien dan 264 PasienRawatUlang) ditambah 984 object RekamMedis.

BAGIAN 4 | DIVA NURISLAMIA AGFA

Enkapsulasi: Atribut Private, Getter & Setter

Slide 13–16 • **Durasi** ± 3,5 menit • **Waktu baca** ± 2 menit 40 detik • **Waktu tampil** ± 3 menit 20 detik • 320 kata

Kata kunci: Enkapsulasi · semua atribut dibuat private · AttributeError · name mangling · getter · setter · validasi · ValueError · 7 · nilai kepuasan tetap 4

SLIDE 13 Diva Nurislamia Agfa

± 5 detik

Terima kasih, Fahmi. Saya Diva, akan menjelaskan enkapsulasi. KLIK →

SLIDE 14 Atribut Private Tidak Bisa Diakses Langsung

± 50 detik

Enkapsulasi artinya data dibungkus di dalam class dan tidak bisa diubah sembarangan dari luar, supaya data penting seperti umur atau kepuasan selalu diperiksa dulu. Di project kami, **semua atribut dibuat private** dengan awalan dua garis bawah, misalnya `__umur`. *(tunjuk: terminal)* Ini percobaan langsung di Python dengan class kami. Perintah pertama membuat object pasien ID 1. *(tunjuk: pasien.kepuasan)* Perintah kedua membaca kepuasan lewat property, hasilnya 4; inilah cara yang benar. *(tunjuk: pasien.__kepuasan)* Perintah ketiga mencoba membaca atribut private secara langsung, hasilnya **AttributeError**, dan Python menyarankan memakai kepuasan. *(tunjuk: daftar nama di bawah)* Perintah terakhir menampilkan nama semua atribut di dalam object. Namanya sudah diubah oleh Python, misalnya `__umur` menjadi `_Orang__umur`. Ini disebut **name mangling**. KLIK →

Penjelasan perintah di sesi Python (terminal di slide)

Perintah	Fungsinya
`>>> pasien = Pasien(1, 45, "Female", 4)`	Membuat object `Pasien` dengan ID 1, umur 45, jenis kelamin Female, dan kepuasan 4.
`>>> pasien.kepuasan`	Membaca kepuasan lewat property (getter); hasilnya 4. Inilah cara yang benar.
`>>> pasien.__kepuasan`	Mencoba membaca atribut private secara langsung; gagal dengan `AttributeError`, dan Python menyarankan kepuasan.
`>>> for nama in vars(pasien): print(nama)`	Mencetak nama semua atribut yang tersimpan di object; terlihat hasil name mangling seperti `_Orang__umur`.

SLIDE 15 Setter Memvalidasi Setiap Nilai Sebelum Disimpan

± 1 menit

Lalu bagaimana membaca dan mengubah atribut private? Ini kodenya untuk umur. *(tunjuk: nomor 1, baris 16 sampai 18)* Baris 16, `@property`, menjadikan method di bawahnya sebagai **getter**. Baris 17 dan 18 berfungsi membaca umur, yaitu mengembalikan nilai dari `__umur`. *(tunjuk: nomor 2, baris 20 dan 21)* Baris 20, `@umur.setter`, menjadikan method di bawahnya sebagai **setter**, yang dipanggil setiap kali kita menulis pasien titik umur sama dengan suatu

nilai. *(tunjuk: nomor 3, baris 23 dan 24)* Baris 23 adalah **validasi**: memeriksa apakah nilainya bilangan bulat dan lebih dari nol. Kalau tidak, baris 24 memunculkan **ValueError**, sehingga nilai yang salah tidak tersimpan. *(tunjuk: baris 25)* Kalau lolos, baris 25 menyimpan nilainya ke `__umur`. *(tunjuk: kotak aturan)* Pola yang sama kami pakai di semua setter, misalnya kepuasan satu sampai lima dan lama rawat minimal satu hari. Constructor juga memakai setter, jadi data dari CSV langsung divalidasi sejak object dibuat. **KLIK →**

Penjelasan kode · orang.py baris 16–25

Baris	Kode	Fungsinya
16	`@property`	`@property` menjadikan method di bawahnya sebagai getter, sehingga bisa dibaca seperti atribut: `orang.umur`.
17	`def umur(self):`	Getter `umur`: dipanggil saat kode membaca `orang.umur`.
18	`return self.__umur`	Mengembalikan nilai dari atribut private `__umur`.
20	`@umur.setter`	`@umur.setter` menjadikan method di bawahnya sebagai setter untuk umur.
21	`def umur(self, nilai):`	Setter `umur`: dipanggil saat kode menulis `orang.umur = nilai`.
22	`# Aturan data: Age harus bilangan positif`	Komentar: aturan data, umur harus bilangan positif.
23	`if not isinstance(nilai, int) or nilai <= 0:`	Validasi: menolak nilai yang bukan bilangan bulat (`int`) atau yang kurang dari/sama dengan 0.
24	`raise ValueError(f"Umur harus bilangan bulat positif, bukan {nilai}")`	Memunculkan `ValueError` beserta pesan, sehingga nilai salah tidak pernah disimpan.
25	`self.__umur = nilai`	Jika lolos validasi, nilai disimpan ke atribut private `__umur`.

SLIDE 16 Data Tidak Valid Ditolak, Data Lama Tetap Aman

± 45 detik

Sekarang buktinya, dari fungsi demo di `main.py`. *(tunjuk: baris 139)* Baris 139, `try`, menandai kode yang mungkin menimbulkan error. *(tunjuk: baris 140)* Baris 140 sengaja mengisi kepuasan pasien ID 1 dengan **7**, padahal aturannya satu sampai lima. *(tunjuk: baris 141 dan 142)* Baris 141, `except ValueError`, menangkap error dari setter supaya program tidak berhenti, dan baris 142 mencetak pesan errornya. *(tunjuk: output)* Di output terlihat pesan ValueError, dan di baris terakhir **nilai kepuasan tetap 4**. Tanpa enkapsulasi, nilai tujuh itu akan langsung tersimpan dan merusak perhitungan, misalnya rata-rata kepuasan. Jadi manfaat utama enkapsulasi adalah data selalu dalam keadaan valid. *(jeda)* Selanjutnya, **Rosa** akan menjelaskan inheritance. **KLIK →**

Penjelasan kode · main.py baris 139–142

Baris	Kode	Fungsinya
139	`try:`	`try`: menandai kode yang mungkin menimbulkan error.
140	`pasien.kepuasan = 7  # tidak valid, karena aturan kepuasan 1 sampai 5`	Sengaja mengisi kepuasan dengan 7 (tidak valid, aturannya 1–5); setter akan menolak.
141	`except ValueError as error:`	`except ValueError`: menangkap error dari setter supaya program tidak berhenti.
142	`print(f"ValueError tertangkap: {error}")`	Mencetak pesan error yang tertangkap.

> **Kalimat penutup bagianmu:** “Selanjutnya, Rosa akan menjelaskan inheritance.”

Persiapan tanya jawab

T **Apa yang terjadi jika kita menulis print(pasien.__kepuasan)?**

J Muncul AttributeError karena nama atribut private sudah diganti oleh Python (name mangling). Cara yang benar adalah pasien.kepuasan, yaitu lewat property.

T **Kenapa constructor memanggil setter, bukan langsung mengisi self.__umur?**

J Supaya validasi juga berlaku saat object pertama kali dibuat. Jadi data CSV yang salah langsung tertolak.

T **Kenapa memakai ValueError?**

J Karena yang salah adalah nilai datanya. ValueError adalah exception bawaan Python untuk nilai yang tidak sesuai, dan bisa ditangkap dengan try-except.

BAGIAN 5 | ROSA RAMADANI

Inheritance (1): Pasien Mewarisi Orang

Slide 17–19 • **Durasi** ± 3 menit • **Waktu baca** ± 2 menit 20 detik • **Waktu tampil** ± 2 menit 50 detik • 276 kata

Kata kunci: Inheritance · class Pasien mewarisi class Orang · super().__init__ · constructor chaining · method overriding · super().tampilkan_info() · memakai ulang · menambah

SLIDE 17 Rosa Ramadani

± 5 detik

Terima kasih, Diva. Saya Rosa, akan menjelaskan inheritance bagian pertama. KLIK →

SLIDE 18 Pasien Mewarisi Atribut dan Validasi dari Orang

± 1 menit 10 detik

Inheritance atau pewarisan artinya class anak mendapatkan atribut dan method dari class induknya, seperti anak yang mewarisi sifat orang tuanya lalu punya ciri khasnya sendiri. *(tunjuk: diagram)* Di project kami, **class Pasien mewarisi class Orang**; panah bersegitiga ini berarti mewarisi. Jadi Pasien otomatis punya umur dan jenis kelamin beserta validasinya, dan hanya menambah patient_id serta kepuasan. Keuntungannya, kalau aturan validasi umur diubah di Orang, perubahan itu otomatis berlaku untuk semua pasien. Sekarang kodenya. *(tunjuk: baris 9)* Baris 9, `class Pasien(Orang)`, fungsinya membuat class Pasien yang mewarisi Orang; nama induknya ditulis di dalam kurung. *(tunjuk: baris 12)* Baris 12 adalah constructor Pasien yang menerima empat data. *(tunjuk: baris 14)* Baris 14, **super().__init__**, menjalankan constructor milik Orang untuk mengisi umur dan jenis kelamin. Ini disebut **constructor chaining**. Kalau baris ini lupa ditulis, atribut umur dan jenis kelamin tidak pernah dibuat dan program akan error. *(tunjuk: baris 16 dan 17)* Baris 16 dan 17 mengisi patient_id dan kepuasan lewat setter milik Pasien. KLIK →

Penjelasan kode · pasien.py baris 9–17

Baris	Kode	Fungsinya
9	`class Pasien(Orang):`	Membuat class `Pasien` yang mewarisi `Orang` (nama induk ditulis di dalam kurung).
10	`"""Pasien rumah sakit (turunan dari Orang)."""`	Docstring: keterangan class.
12	`def __init__(self, patient_id, umur, jenis_kelamin, kepuasan):`	Constructor `Pasien`: menerima empat data, yaitu ID, umur, jenis kelamin, dan kepuasan.
13	`# [INHERITANCE] Constructor chaining: umur & jenis_kelamin diurus oleh Orang`	Komentar: umur & jenis kelamin diurus oleh class Orang.
14	`super().__init__(umur, jenis_kelamin)`	`super().__init__(...)` menjalankan constructor milik `Orang` (constructor chaining) untuk mengisi umur dan jenis kelamin.
15	`# [ENKAPSULASI] Diisi lewat setter agar tervalidasi`	Komentar: atribut tambahan diisi lewat setter.
16	`self.patient_id = patient_id`	Mengisi `patient_id` lewat setter-nya (harus bilangan bulat positif).
17	`self.kepuasan = kepuasan`	Mengisi `kepuasan` lewat setter-nya (harus 1 sampai 5).

SLIDE 19 Overriding: Pakai Ulang Method Induk, Lalu Tambahkan

± 1 menit

Selain mewarisi, Pasien juga melakukan **method overriding**, yaitu menulis ulang method milik induknya dengan nama yang sama. *(tunjuk: baris 42)* Baris 42 membuat ulang method tampilkan_info di class Pasien. *(tunjuk: baris 44)* Baris 44 mencetak ID pasien. *(tunjuk: baris 45)* Baris 45 memanggil versi milik Orang lewat **super().tampilkan_info()**, sehingga umur dan jenis kelamin ikut tercetak tanpa menyalin kode. *(tunjuk: baris 46)* Baris 46 mencetak kepuasan dengan format per lima. *(tunjuk: output)* Hasilnya terlihat di sini pada pasien pertama dari data asli: baris umur dan jenis kelamin berasal dari Orang, sedangkan ID pasien dan kepuasan ditambahkan oleh Pasien. Jadi overriding membuat kita bisa **memakai ulang** kode induk sekaligus **menambah** perilaku baru. Perlu dibedakan, overriding tidak sama dengan overloading: overriding menulis ulang method induk di class anak dengan nama yang sama. *(jeda)* Selanjutnya, **Sofiang** akan menjelaskan pewarisan bertingkat dan polimorfisme. **KLIK →**

Penjelasan kode · pasien.py baris 41–46

Baris	Kode	Fungsinya
41	`# ---------- [INHERITANCE] Method overriding ----------`	Komentar penanda method overriding.
42	`def tampilkan_info(self):`	Menulis ulang (override) method `tampilkan_info()` milik `Orang`.
43	`"""Override tampilkan_info() milik Orang: tambah ID pasien dan kepuasan."""`	Docstring: keterangan method override.
44	`print(f"ID Pasien : {self.patient_id}")`	Mencetak ID pasien.
45	`super().tampilkan_info()  # memakai ulang tampilan umur & jenis kelamin dari Orang`	Memanggil `tampilkan_info()` milik `Orang` lewat `super()`, sehingga umur dan jenis kelamin ikut tercetak.
46	`print(f"Kepuasan : {self.kepuasan}/5")`	Mencetak kepuasan dalam format skala 5, misalnya `4/5`.

Kalimat penutup bagianmu: “Selanjutnya, Sofiang akan menjelaskan pewarisan bertingkat dan polimorfisme.”

Persiapan tanya jawab

T **Apa yang terjadi jika super().__init__() tidak dipanggil?**

J Constructor Orang tidak berjalan, jadi atribut umur dan jenis_kelamin tidak pernah dibuat. Saat diakses akan muncul AttributeError.

T **Apa itu method overriding?**

J Class anak menulis ulang method dengan nama yang sama milik class induk untuk mengubah atau menambah perilakunya, seperti tampilkan_info() di Pasien.

T **Apa bedanya overriding dan overloading?**

J Overriding = menulis ulang method induk di class anak. Overloading = beberapa method bernama sama dengan parameter berbeda dalam satu class, yang tidak didukung langsung oleh Python seperti di Java.

BAGIAN 6 | SOFIANG

Inheritance (2): Pewarisan Bertingkat & Polimorfisme

Slide 20–22 • **Durasi** ± 2,5 menit • **Waktu baca** ± 2 menit 10 detik • **Waktu tampil** ± 2 menit 40 detik • 260 kata

Kata kunci: bertingkat · PasienRawatUlang · 264 pasien · perilaku khusus · status_rawat · Status Rawat · Status Rawat: Pasien rawat ulang · polimorfisme · MRO

SLIDE 20 Sofiang

± 5 detik

Terima kasih, Rosa. Saya Sofiang, melanjutkan inheritance bagian kedua. **KLIK →**

SLIDE 21 PasienRawatUlang Mewarisi Pasien dan Orang Sekaligus

± 1 menit 10 detik

Inheritance di project kami **bertingkat**. *(tunjuk: diagram dari atas ke bawah)* Orang adalah induk dari Pasien, dan Pasien adalah induk dari **PasienRawatUlang**. Jadi PasienRawatUlang mewarisi semua milik Pasien dan Orang sekaligus, dan satu object PasienRawatUlang juga dianggap sebagai Pasien dan sebagai Orang. Class ini dipakai untuk **264 pasien** yang Readmission-nya Yes. Sekarang kodenya. *(tunjuk: baris 10)* Baris 10, `class PasienRawatUlang(Pasien)`, membuat class yang mewarisi Pasien. *(tunjuk: baris 13 dan 15)* Baris 13 adalah constructor-nya, dan baris 15 memanggil super().__init__ untuk meneruskan semua data ke Pasien, lalu Pasien meneruskan umur dan jenis kelamin ke Orang. *(tunjuk: baris 18 sampai 20)* Class ini tidak menambah atribut, karena data CSV tidak punya kolom tambahan, tetapi punya **perilaku khusus**. Baris 18 membuat method **status_rawat**, dan baris 20 mengembalikan teks Pasien rawat ulang. Method ini tidak dimiliki Pasien maupun Orang. *(tunjuk: baris 23 sampai 26)* Baris 23 meng-override tampilkan_info. Baris 25 memanggil versi milik Pasien lewat super, dan baris 26 menambahkan baris **Status Rawat** yang isinya diambil dari status_rawat. **KLIK →**

Penjelasan kode · pasien_rawat_ulang.py baris 10–26

Baris	Kode	Fungsinya
10	`class PasienRawatUlang(Pasien):`	Membuat class `PasienRawatUlang` yang mewarisi `Pasien` (pewarisan bertingkat: Orang → Pasien → PasienRawatUlang).
11	`"""Pasien yang kembali dirawat (Readmission = Yes)."""`	Docstring: class ini dipakai untuk baris data dengan Readmission = Yes.
13	`def __init__(self, patient_id, umur, jenis_kelamin, kepuasan):`	Constructor dengan empat data yang sama seperti `Pasien`.
14	`# [INHERITANCE] Constructor chaining: Pasien -> Orang`	Komentar: rantai constructor dari Pasien ke Orang.
15	`super().__init__(patient_id, umur, jenis_kelamin, kepuasan)`	Meneruskan semua data ke constructor `Pasien`, yang lalu meneruskan umur & jenis kelamin ke `Orang`.
17	`# ---------- Method khusus PasienRawatUlang ----------`	Komentar penanda method khusus, yaitu method yang hanya dimiliki `PasienRawatUlang`.
18	`def status_rawat(self):`	Method `status_rawat()`: perilaku khas `PasienRawatUlang`, tidak ada di `Pasien` maupun `Orang`.
19	`"""Mengembalikan keterangan status rawat pasien ini."""`	Docstring: keterangan method.

Baris	Kode	Fungsinya
20	`return "Pasien rawat ulang"`	Mengembalikan teks “Pasien rawat ulang” sebagai keterangan status pasien ini.
22	`# ---------- [INHERITANCE] Method overriding ----------`	Komentar penanda method overriding.
23	`def tampilkan_info(self):`	Override `tampilkan_info()` milik `Pasien`.
24	`"""Override tampilkan_info() milik Pasien: tambah status_rawat()."""`	Docstring: keterangan method override.
25	`super().tampilkan_info()  # tampilkan ID, umur, jenis kelamin, kepuasan`	Memanggil versi `Pasien` lewat `super()` untuk mencetak ID, umur, jenis kelamin, dan kepuasan.
26	`print(f"Status Rawat   : {self.status_rawat()}")`	Menambahkan baris `Status Rawat`, isinya diambil dari method `status_rawat()`.

SLIDE 22 Satu Method, Hasil Berbeda: Polimorfisme

± 55 detik

Hasil overriding itu terlihat di sini. *(tunjuk: terminal kiri)* Rekam pertama adalah object Pasien, tampilannya berhenti di kepuasan. *(tunjuk: terminal kanan)* Rekam kedua adalah object PasienRawatUlang, dan ada tambahan baris **Status Rawat: Pasien rawat ulang**. *(tunjuk: baris 98)* Padahal di class RekamMedis, baris 98 hanya berisi satu perintah, `self.pasien.tampilkan_info()`, yang fungsinya memanggil tampilkan_info milik object pasien yang tersimpan di rekam medis. Baris 96 dan 97 di atasnya hanya komentar. Python otomatis memilih versi method sesuai class object-nya. Inilah **polimorfisme**: satu perintah, hasilnya berbeda. Tanpa polimorfisme, kita harus menulis pengecekan if untuk setiap jenis pasien. Urutan Python mencari method disebut **MRO**: dimulai dari PasienRawatUlang, lalu Pasien, lalu Orang, sehingga versi PasienRawatUlang yang dipakai lebih dulu. *(jeda)* Selanjutnya, **Revalinda** akan menjelaskan class RekamMedis. **KLIK →**

Penjelasan kode · rekam_medis.py baris 96–98

Baris	Kode	Fungsinya
96	`# Polimorfisme: versi tampilkan_info() yang dipanggil`	Komentar: penjelasan polimorfisme.
97	`# menyesuaikan class object-nya (Pasien atau PasienRawatUlang).`	Komentar lanjutan: versi method mengikuti class object-nya.
98	`self.pasien.tampilkan_info()`	Memanggil `tampilkan_info()` milik object pasien; Python memilih versi `Pasien` atau `PasienRawatUlang` (polimorfisme).

Kalimat penutup bagianmu: “Selanjutnya, Revalinda akan menjelaskan class RekamMedis.”

Persiapan tanya jawab

T **Kenapa PasienRawatUlang tidak punya atribut baru?**

J Karena aturan project melarang atribut di luar kolom data. Yang membedakannya adalah perilaku: method khusus status_rawat() dan override tampilkan_info() yang menampilkan statusnya.

T **Kenapa pasien biasa tidak menampilkan baris Status Rawat?**

J Karena baris itu hanya ditambahkan oleh override tampilkan_info() di class PasienRawatUlang, dan method status_rawat() hanya ada di class itu. Inilah efek overriding.

T **Apa itu MRO?**

J Method Resolution Order, yaitu urutan Python mencari method: PasienRawatUlang → Pasien → Orang → object.

BAGIAN 7 | REVALINDA

Komposisi & Validasi: class RekamMedis

Slide 23–26 • **Durasi** ± 3,5 menit • **Waktu baca** ± 2 menit 40 detik • **Waktu tampil** ± 3 menit 20 detik • 323 kata

Kata kunci: RekamMedis tidak mewarisi class apa pun · komposisi · memiliki · isinstance · data medis juga dijaga oleh setter · ValueError · hitung_biaya_per_hari · tiga ribu · tampilkan_rekam

SLIDE 23 Revalinda

± 5 detik

Terima kasih, Sofiang. Saya Revalinda, akan menjelaskan komposisi pada class RekamMedis. **KLIK →**

SLIDE 24 RekamMedis Memiliki Pasien, Bukan Mewarisinya

± 1 menit

Berbeda dari tiga class sebelumnya, **RekamMedis tidak mewarisi class apa pun**. *(tunjuk: kartu perbandingan)* Inheritance adalah hubungan adalah: Pasien adalah Orang. Sedangkan RekamMedis memakai **komposisi**, yaitu hubungan memiliki: rekam medis **memiliki** data pasien. Kalimat rekam medis adalah pasien tidak masuk akal, sedangkan rekam medis memiliki pasien masuk akal; ini cara mudah memilih antara keduanya. *(tunjuk: diagram kanan)* Di dalam object RekamMedis tersimpan object Pasien, ditambah kondisi, prosedur, biaya, lama rawat, dan hasil. Sekarang kodenya, yaitu setter untuk atribut pasien. *(tunjuk: baris 26 dan 27)* Baris 26, `@pasien.setter`, menandai setter, dan baris 27 menerima nilai baru. *(tunjuk: baris 29)* Baris 29 memeriksa dengan **isinstance**: apakah nilai itu object Pasien. *(tunjuk: baris 30)* Kalau bukan, baris 30 memunculkan ValueError. *(tunjuk: baris 31)* Kalau benar, baris 31 menyimpannya ke atribut private `__pasien`. Karena PasienRawatUlang adalah turunan Pasien, object PasienRawatUlang juga diterima. **KLIK →**

Penjelasan kode · rekam_medis.py baris 26–31

Baris	Kode	Fungsinya
26	`@pasien.setter`	`@pasien.setter` menandai setter untuk atribut pasien.
27	`def pasien(self, nilai):`	Setter `pasien`: dipanggil saat kode menulis `rekam.pasien = nilai`.
28	`# Menerima object Pasien maupun turunannya (PasienRawatUlang)`	Komentar: object Pasien maupun turunannya diterima.
29	`if not isinstance(nilai, Pasien):`	Validasi `isinstance`: memeriksa apakah nilai adalah object `Pasien` (termasuk `PasienRawatUlang`).
30	`raise ValueError("Pasien harus berupa object dari class Pasien")`	Jika bukan object Pasien, memunculkan `ValueError`.
31	`self.__pasien = nilai`	Jika lolos, object disimpan ke atribut private `__pasien`.

SLIDE 25 Data Medis Dijaga Setter: Kosong dan Nol Ditolak

± 50 detik

Selain pasien, **data medis juga dijaga oleh setter**. *(tunjuk: nomor 1, baris 37 sampai 42)* Ini setter kondisi. Baris 39 adalah komentar aturannya: Condition tidak boleh kosong. Baris 40 memeriksa apakah nilainya teks dan tidak kosong; `strip` membuang spasi, jadi isian yang hanya berisi spasi juga dianggap kosong. Kalau kosong, baris 41 memunculkan **ValueError**; kalau lolos, baris 42 menyimpannya. Setter prosedur memakai pola yang sama di baris 48 sampai 53. *(tunjuk: nomor 2, baris 70 sampai 75)* Untuk lama rawat, baris 73 menolak nilai yang bukan bilangan bulat atau kurang dari satu, lalu baris 74 memunculkan ValueError. *(tunjuk: tabel kanan)* Jadi setiap data medis punya aturan, dan *(tunjuk: kotak kuning)* baris CSV yang melanggarnya akan dilewati oleh main.py. KLIK →

Penjelasan kode · rekam_medis.py baris 37–42

Baris	Kode	Fungsinya
37	`@kondisi.setter`	`@kondisi.setter` menandai setter untuk kondisi.
38	`def kondisi(self, nilai):`	Setter `kondisi`: dipanggil saat kode menulis `rekam.kondisi = nilai`.
39	`# Aturan data: Condition tidak boleh kosong`	Komentar: aturan data, Condition tidak boleh kosong.
40	`if not isinstance(nilai, str) or nilai.strip() == "":`	Validasi: nilai harus teks (`str`) dan tidak kosong; `strip()` membuang spasi di awal dan akhir, jadi isian yang hanya berisi spasi juga dianggap kosong.
41	`raise ValueError("Kondisi tidak boleh kosong")`	Jika kosong, memunculkan `ValueError` dengan pesan “Kondisi tidak boleh kosong”.
42	`self.__kondisi = nilai`	Jika lolos, nilai disimpan ke atribut private `__kondisi`.

Penjelasan kode · rekam_medis.py baris 70–75

Baris	Kode	Fungsinya
70	`@lama_rawat.setter`	`@lama_rawat.setter` menandai setter untuk lama rawat.
71	`def lama_rawat(self, nilai):`	Setter `lama_rawat`: dipanggil saat kode menulis `rekam.lama_rawat = nilai`.
72	`# Aturan data: Length_of_Stay bulat > 0 (mencegah pembagian nol)`	Komentar: aturan data, Length_of_Stay bilangan bulat lebih dari 0, sekaligus mencegah pembagian dengan nol.
73	`if not isinstance(nilai, int) or nilai <= 0:`	Validasi: menolak nilai yang bukan bilangan bulat (`int`) atau yang kurang dari/sama dengan 0.
74	`raise ValueError(f"Lama rawat harus bilangan bulat positif, bukan {nilai}")`	Memunculkan `ValueError` beserta nilai yang salah.
75	`self.__lama_rawat = nilai`	Jika lolos, nilai disimpan ke atribut private `__lama_rawat`.

Kode terkait (dibahas, tidak tampil di slide) · rekam_medis.py baris 48–53

Baris	Kode	Fungsinya
48	`@prosedur.setter`	`@prosedur.setter` menandai setter untuk prosedur.
49	`def prosedur(self, nilai):`	Setter `prosedur`: dipanggil saat kode menulis `rekam.prosedur = nilai`.
50	`# Aturan data: Procedure tidak boleh kosong`	Komentar: aturan data, Procedure tidak boleh kosong.
51	`if not isinstance(nilai, str) or nilai.strip() == "":`	Validasi yang sama seperti kondisi: harus teks dan tidak kosong setelah spasi dibuang.

Baris	Kode	Fungsinya
52	`raise ValueError("Prosedur tidak boleh kosong")`	Jika kosong, memunculkan `ValueError` dengan pesan “Prosedur tidak boleh kosong”.
53	`self.__prosedur = nilai`	Jika lolos, nilai disimpan ke atribut private `__prosedur`.

SLIDE 26 Biaya per Hari Dihitung Aman, Tanpa Pembagian Nol

± 50 detik

RekamMedis punya dua method. *(tunjuk: baris 89 sampai 91)* Baris 89 membuat method **hitung_biaya_per_hari**, baris 90 adalah docstring keterangannya, dan baris 91 mengembalikan hasil biaya dibagi lama rawat. Perhitungannya memakai property biaya dan lama_rawat, jadi tetap mengikuti enkapsulasi. *(tunjuk: kotak rumus)* Contohnya pasien ID 1: biayanya lima belas ribu dan dirawat lima hari, jadi biaya per harinya **tiga ribu**. *(tunjuk: tabel)* Tabel di kanan menunjukkan hasilnya untuk lima pasien pertama. *(tunjuk: kotak kuning)* Pembagiannya aman, karena tadi setter lama rawat sudah menolak nilai di bawah satu hari. *(tunjuk: kartu tampilkan_rekam)* Method kedua, **tampilkan_rekam**, menampilkan data pasien lewat tampilkan_info milik object pasien, lalu data medisnya. *(jeda)* Selanjutnya, **Andi** akan menjelaskan program utama di main.py. **KLIK →**

Penjelasan kode · rekam_medis.py baris 88–91

Baris	Kode	Fungsinya
88	`# ---------- Method ----------`	Komentar penanda bagian method.
89	`def hitung_biaya_per_hari(self):`	Method `hitung_biaya_per_hari()`.
90	`"""Menghitung biaya rata-rata untuk setiap hari rawat."""`	Docstring: keterangan method.
91	`return self.biaya / self.lama_rawat`	Mengembalikan biaya dibagi lama rawat; aman karena lama rawat minimal 1 hari.

Kalimat penutup bagianmu: “Selanjutnya, Andi akan menjelaskan program utama di main.py.”

Persiapan tanya jawab

T Kenapa RekamMedis tidak mewarisi Pasien?

J Karena rekam medis bukan jenis pasien. Hubungannya “memiliki” (has-a), bukan “adalah” (is-a), sehingga yang tepat adalah komposisi.

T Bagaimana jika lama_rawat bernilai 0?

J Setter lama_rawat menolaknya dengan ValueError, sehingga hitung_biaya_per_hari() tidak pernah membagi dengan nol.

T Kenapa kondisi yang hanya berisi spasi juga ditolak?

J Karena setter memeriksa nilai.strip() == "". strip() membuang spasi di awal dan akhir, jadi teks yang isinya hanya spasi dianggap kosong.

T Kenapa setter pasien bisa menerima PasienRawatUlang?

J Karena PasienRawatUlang adalah turunan Pasien, sehingga isinstance(nilai, Pasien) bernilai True.

BAGIAN 8 | ANDI REZKY NABILA

Program Utama (main.py): Membaca CSV & Menu Interaktif

Slide 27–30 • **Durasi** ± 3,5 menit • **Waktu baca** ± 2 menit 45 detik • **Waktu tampil** ± 3 menit 25 detik • 330 kata

Kata kunci: main.py · menu · baris_dilewati · DictReader · mencatat nomor barisnya · program tetap berjalan · menu interaktif · break

SLIDE 27 Andi Rezky Nabila

± 5 detik

Terima kasih, Revalinda. Saya Andi, akan menjelaskan program utama di main.py, termasuk menu interaktifnya. KLIK →

SLIDE 28 main.py Mengatur Alur Program dari Awal sampai Akhir

± 50 detik

Semua class tadi dijalankan dari **main.py**. *(tunjuk: flowchart dari atas)* Program mencari file CSV lebih dulu; kalau tidak ada, program berhenti dengan pesan. Kalau ada, setiap baris diubah menjadi object lalu dicek. Baris yang valid disimpan ke list, yang tidak valid dilewati dan dicatat nomornya. *(tunjuk: kolom kanan flowchart)* Setelah itu program menampilkan output awal, lalu masuk ke **menu** yang berulang sampai pengguna memilih keluar. *(tunjuk: daftar fungsi)* Setiap langkah punya fungsinya sendiri: `cari_file_csv` mencari file, `muat_data` membaca CSV, `buat_pasien` dan `buat_rekam_medis` membuat object, tiga fungsi berikutnya menampilkan output awal, dan `jalankan_menu` menjalankan menu. Semuanya dipanggil berurutan oleh fungsi `main`, yang hanya berjalan jika file ini dijalankan langsung, lewat pengecekan `if __name__` di baris paling bawah. KLIK →

Penjelasan fungsi-fungsi di main.py

Fungsi	Baris	Fungsinya
`cari_file_csv()`	27–33	Mencari lokasi file CSV di folder main.py (dua variasi nama); mengembalikan `None` jika tidak ada.
`buat_pasien()`	36–50	Mengubah satu baris CSV menjadi object `Pasien` atau `PasienRawatUlang` berdasarkan kolom Readmission.
`buat_rekam_medis()`	53–63	Membuat object `RekamMedis` berisi object pasien dan data medis dari baris CSV.
`muat_data()`	66–81	Membaca seluruh CSV, membuat object untuk tiap baris, melewati dan mencatat baris tidak valid, lalu mengembalikan list object dan list nomor baris yang dilewati.
`tampilkan_rekam_pertama()`	84–90	Mencetak 5 rekam medis pertama beserta nama class object pasiennya.
`tampilkan_ringkasan()`	93–130	Menghitung dan mencetak total pasien, rawat ulang, rata-rata biaya, rata-rata lama rawat, dan kondisi terbanyak.
`demo_validasi_setter()`	133–143	Mendemokan setter yang menolak kepuasan 7 di dalam try-except.
`menu_lihat_data()`	163–174	Menu 1: menampilkan 10 data pertama dalam tabel singkat.
`cari_rekam()`	177–182	Mencari rekam medis berdasarkan Patient_ID; mengembalikan `None` jika tidak ada.

Fungsi	Baris	Fungsinya
`menu_cari_pasien()`	185–198	Menu 2: membaca Patient_ID, mencarinya, lalu menampilkan rekam medisnya.
`menu_tambah_pasien()`	201–218	Menu 3: membaca data pasien baru, memeriksanya lewat setter yang sama dengan data CSV, lalu menambahkannya ke list.
`jalankan_menu()`	221–240	Menampilkan menu berulang (`while True`) sampai pengguna memilih 4 (Keluar).
`main()`	243–267	Fungsi utama yang menjalankan semua langkah di atas secara berurutan.
`if __name__ == "__main__":`	270–271	Menjalankan `main()` hanya jika file dijalankan langsung dengan `python main.py`.

SLIDE 29 Baris CSV Rusak Dilewati dan Dicatat, Program Tetap Jalan

± 1 menit 5 detik

Sekarang isi fungsi muat_data. *(tunjuk: baris 69)* Baris 69 menyiapkan list **baris_dilewati** untuk mencatat nomor baris yang tidak valid. *(tunjuk: baris 71)* Baris 71 membuka file CSV; `with` menutup file otomatis, dan utf-8-sig aman untuk file yang pernah disimpan lewat Excel. *(tunjuk: nomor 1, baris 73)* Baris 73 membuat **DictReader**, yang membaca setiap baris sebagai dictionary berdasarkan nama kolom. `restval` diisi teks kosong, jadi baris yang kolomnya kurang ikut ditolak validasi, bukan membuat program berhenti. *(tunjuk: nomor 2, baris 74 sampai 76)* Baris 74 mengulang setiap baris, baris 75 mencoba memprosesnya, dan baris 76 menambahkan object ke list. *(tunjuk: nomor 3, baris 77 sampai 80)* Kalau data melanggar aturan, baris 77 menangkap ValueError, baris 79 **mencatat nomor barisnya**, dan baris 80 mencetak peringatan, lalu **program tetap berjalan**. *(tunjuk: baris 81)* Baris 81 mengembalikan list object dan list baris yang dilewati. Kami sudah mengujinya dengan baris rusak, dan untuk data asli kami, baris yang dilewati nol. KLIK →

Penjelasan kode · main.py baris 68–81

Baris	Kode	Fungsinya
68	`daftar_rekam = []`	Menyiapkan list kosong untuk menampung semua object.
69	`baris_dilewati = []  # nomor baris CSV yang tidak valid`	Menyiapkan list `baris_dilewati` untuk mencatat nomor baris CSV yang tidak valid.
70	`# utf-8-sig: tetap aman jika CSV pernah disimpan ulang lewat Excel (ada BOM)`	Komentar: alasan memakai encoding utf-8-sig.
71	`with open(lokasi_file, mode="r", encoding="utf-8-sig", newline="") as file:`	Membuka file CSV; `with` memastikan file ditutup otomatis, dan utf-8-sig aman untuk file yang disimpan lewat Excel.
72	`# restval="": kolom yang hilang dianggap teks kosong, jadi ikut ditolak validasi`	Komentar: alasan memakai `restval=""`.
73	`pembaca = csv.DictReader(file, restval="")`	`csv.DictReader` membaca setiap baris sebagai dictionary berdasarkan nama kolom, misalnya `baris["Age"]`. `restval=""` mengisi kolom yang hilang dengan teks kosong, sehingga baris yang kolomnya kurang ditolak validasi, bukan menghentikan program.
74	`for baris in pembaca:`	Mengulang setiap baris data di file.
75	`try:`	`try`: mencoba membuat object dari baris ini.
76	`daftar_rekam.append(buat_rekam_medis(baris))`	Membuat object rekam medis lalu menambahkannya ke list dengan append.

Baris	Kode	Fungsinya
77	`except ValueError as error:`	`except ValueError`: menangkap error dari setter (atau dari `int()`/`float()`) jika data melanggar aturan.
78	`# Baris yang melanggar aturan dilewati dan dicatat, program tetap berjalan`	Komentar: baris yang salah dilewati dan dicatat, program tetap berjalan.
79	`baris_dilewati.append(pembaca.line_num)`	Mencatat nomor baris yang sedang dibaca (`pembaca.line_num`) ke list `baris_dilewati`.
80	`print(f"[PERINGATAN] Baris {pembaca.line_num} dilewati: {error}")`	Mencetak peringatan beserta nomor baris dan alasannya.
81	`return daftar_rekam, baris_dilewati`	Mengembalikan dua hasil: list object yang valid dan list nomor baris yang dilewati.

Kode terkait (dibahas, tidak tampil di slide) · main.py baris 253–258

Baris	Kode	Fungsinya
253	`daftar_rekam, baris_dilewati = muat_data(lokasi_file)`	Memanggil `muat_data()` dan menerima dua hasil: list object dan list nomor baris yang dilewati.
254	`print(f"File data : {os.path.basename(lokasi_file)}")`	Mencetak nama file data yang dipakai.
255	`print(f"Object RekamMedis : {len(daftar_rekam)} object tersimpan di list")`	Mencetak jumlah object RekamMedis yang tersimpan di list.
256	`print(f"Baris dilewati : {len(baris_dilewati)}")`	Mencetak jumlah baris yang dilewati.
257	`if len(baris_dilewati) > 0:`	Jika ada baris yang dilewati …
258	`print(f"Nomor baris CSV : {baris_dilewati}")`	… nomor-nomor barisnya juga dicetak.

SLIDE 30 Menu Interaktif: Berulang sampai Pengguna Keluar

± 40 detik

Bagian terakhir adalah **menu interaktif** di fungsi jalankan_menu. *(tunjuk: baris 223)* Baris 223, `while True`, membuat perulangan, jadi menu muncul lagi setiap kali satu pilihan selesai. *(tunjuk: baris 224 sampai 228)* Baris 224 sampai 228 mencetak judul dan empat pilihan. *(tunjuk: baris 229)* Baris 229 membaca pilihan dengan `input`. *(tunjuk: baris 230 sampai 235)* Lalu if dan elif menjalankan fungsinya: pilihan 1 menu_lihat_data, pilihan 2 menu_cari_pasien, dan pilihan 3 menu_tambah_pasien. *(tunjuk: baris 236 sampai 238)* Pilihan 4 mencetak salam penutup, lalu **break** di baris 238 menghentikan perulangan. *(tunjuk: baris 239 dan 240)* Pilihan lain cukup dijawab dengan pesan, bukan error. *(jeda)* Selanjutnya, **Alfira** akan menunjukkan hasil output program dan demo menunya. **KLIK →**

Penjelasan kode · main.py baris 221–240

Baris	Kode	Fungsinya
221	`def jalankan_menu(daftar_rekam):`	Fungsi `jalankan_menu()`: menampilkan menu interaktif.
222	`"""Menampilkan menu berulang-ulang sampai pengguna memilih Keluar."""`	Docstring: keterangan fungsi.
223	`while True:`	`while True` membuat perulangan tanpa batas; menu tampil lagi setelah setiap pilihan selesai.
224	`cetak_judul("MENU")`	Mencetak judul MENU.
225	`print(f"1. Lihat {JUMLAH_DATA_MENU} data pertama")`	Mencetak pilihan 1 (jumlah datanya diambil dari konstanta `JUMLAH_DATA_MENU`).

Baris	Kode	Fungsinya
226	`print("2. Cari pasien berdasarkan Patient_ID")`	Mencetak pilihan 2.
227	`print("3. Tambah pasien baru")`	Mencetak pilihan 3.
228	`print("4. Keluar")`	Mencetak pilihan 4.
229	`pilihan = input("Pilih menu (1-4): ").strip()`	Membaca pilihan pengguna dengan `input()`; `strip()` membuang spasi yang tidak sengaja terketik.
230	`if pilihan == "1":`	Jika pilihan "1" …
231	`menu_lihat_data(daftar_rekam)`	… menjalankan `menu_lihat_data()`.
232	`elif pilihan == "2":`	Jika pilihan "2" …
233	`menu_cari_pasien(daftar_rekam)`	… menjalankan `menu_cari_pasien()`.
234	`elif pilihan == "3":`	Jika pilihan "3" …
235	`menu_tambah_pasien(daftar_rekam)`	… menjalankan `menu_tambah_pasien()`.
236	`elif pilihan == "4":`	Jika pilihan "4" …
237	`print("Terima kasih. Program selesai.")`	… mencetak salam penutup …
238	`break`	… lalu `break` menghentikan perulangan `while`, sehingga program selesai.
239	`else:`	Selain 1 sampai 4 …
240	`print("Pilihan tidak tersedia. Masukkan angka 1 sampai 4.")`	… mencetak pesan pilihan tidak tersedia, lalu menu tampil lagi.

Kode terkait (dibahas, tidak tampil di slide) · main.py baris 146–160

Baris	Kode	Fungsinya
146	`# ======================== MENU INTERAKTIF ========================`	Komentar penanda awal bagian menu interaktif.
147	`JUMLAH_DATA_MENU = 10`	Konstanta jumlah data yang ditampilkan menu 1 (10).
149	`# Pertanyaan saat menambah pasien baru: (nama kolom CSV, teks pertanyaan)`	Komentar: keterangan daftar pertanyaan untuk menu tambah pasien.
150	`PERTANYAAN_PASIEN_BARU = [`	Membuat list `PERTANYAAN_PASIEN_BARU`; setiap isinya pasangan (nama kolom CSV, teks pertanyaan).
151	`("Age", "Umur"),`	Kolom Age ditanyakan dengan teks “Umur”.
152	`("Gender", "Jenis kelamin (Male/Female)"),`	Kolom Gender; teks pertanyaannya menyebut pilihan yang valid (Male/Female).
153	`("Condition", "Kondisi"),`	Kolom Condition (kondisi).
154	`("Procedure", "Prosedur"),`	Kolom Procedure (prosedur).
155	`("Cost", "Biaya"),`	Kolom Cost (biaya).
156	`("Length_of_Stay", "Lama rawat (hari)"),`	Kolom Length_of_Stay (lama rawat dalam hari).
157	`("Readmission", "Rawat ulang (Yes/No)"),`	Kolom Readmission; jawaban Yes/No menentukan class pasien yang dibuat.
158	`("Outcome", "Hasil (Recovered/Stable)"),`	Kolom Outcome (Recovered/Stable).
159	`("Satisfaction", "Kepuasan (1-5)"),`	Kolom Satisfaction (kepuasan 1–5).
160	`]`	Penutup list.

Kalimat penutup bagianmu: “Selanjutnya, Alfira akan menunjukkan hasil output program dan demo menunya.”

Persiapan tanya jawab

T Kenapa memakai csv.DictReader, bukan csv.reader?

J DictReader membaca baris berdasarkan nama kolom (baris["Age"]), sehingga kode lebih mudah dibaca dan tidak bergantung pada urutan kolom.

T **Apa yang terjadi jika ada baris CSV yang kolomnya kurang?**

J Kolom yang hilang diisi teks kosong karena restval="", lalu ditolak validasi dengan ValueError. Baris itu dilewati dan dicatat, program tetap berjalan.

T **Kenapa menu memakai while True?**

J Supaya menu tampil lagi setiap kali satu pilihan selesai. Perulangan baru berhenti saat pengguna memilih 4, karena ada perintah break.

T **Apa yang terjadi jika file CSV tidak ada?**

J Program menampilkan pesan “File CSV tidak ditemukan!” lalu berhenti dengan rapi, tanpa error panjang.

T **Kenapa memakai encoding utf-8-sig?**

J Supaya file tetap terbaca benar walaupun pernah disimpan ulang lewat Excel, yang kadang menambahkan karakter BOM di awal file.

BAGIAN 9 | ALFIRA YUNITA SYAM

Hasil Output, Demo Menu & Kesimpulan

Slide 31–37 • **Durasi** ± 4 menit • **Waktu baca** ± 3 menit 15 detik • **Waktu tampil** ± 4 menit 5 detik • 394 kata

Kata kunci: 984 object · nol · lima rekam medis pertama · 984 pasien · 264 pasien dirawat ulang · delapan ribu tiga ratus enam puluh tujuh koma empat delapan · tiga puluh tujuh koma enam enam hari · 67 pasien · seri · semua kondisi · sepuluh data pertama · PasienRawatUlang · 1001 · setter yang sama · Class dan object · Enkapsulasi · Inheritance

SLIDE 31 Alfira Yunita Syam

± 5 detik

Terima kasih, Andi. Saya Alfira, akan menunjukkan hasil program, demo menu, dan kesimpulan. KLIK →

SLIDE 32 Program Memuat 984 Object, Menampilkan 5 Rekam Pertama

± 30 detik

Ini hasil saat program dijalankan. *(tunjuk: baris Object RekamMedis)* Program memuat **984 object**, dan baris yang dilewati **nol**, artinya semua data asli valid. *(tunjuk: tabel kanan)* Agar ringkas, hanya **lima rekam medis pertama** yang ditampilkan: di fungsi `tampilkan_rekam_pertama`, `daftar_rekam[:JUMLAH_DITAMPILKAN]` mengambil lima object pertama, dan `type(rekam.pasien).__name__` mengambil nama class-nya, sehingga terlihat rekam kedua dan keempat adalah PasienRawatUlang. Seluruh 984 data tetap dipakai untuk menghitung ringkasan. KLIK →

Kode terkait (dibahas, tidak tampil di slide) · main.py baris 84–90

Baris	Kode	Fungsinya
84	`def tampilkan_rekam_pertama(daftar_rekam):`	Fungsi `tampilkan_rekam_pertama()`.
85	`cetak_judul(f"{JUMLAH_DITAMPILKAN} REKAM MEDIS PERTAMA (dari {len(daftar_rekam)} data)")`	Mencetak judul, termasuk jumlah seluruh data.
86	`for nomor, rekam in enumerate(daftar_rekam[:JUMLAH_DITAMPILKAN], start=1):`	`daftar_rekam[:JUMLAH_DITAMPILKAN]` mengambil 5 object pertama (konstanta bernilai 5); `enumerate` memberi nomor urut mulai 1.
87	`# type(object).__name__ = nama class tempat object tersebut dibuat`	Komentar: cara mendapatkan nama class sebuah object.
88	`nama_class = type(rekam.pasien).__name__`	Mengambil nama class object pasien (`Pasien` atau `PasienRawatUlang`).
89	`print(f"\n[{nomor}] Object pasien dari class: {nama_class}")`	Mencetak nomor rekam dan nama class-nya.
90	`rekam.tampilkan_rekam()`	Memanggil `tampilkan_rekam()` untuk mencetak isi rekam medis.

SLIDE 33 Ringkasan Data dari 984 Pasien

± 40 detik

(tunjuk: kartu satu per satu) Totalnya ada **984 pasien**. Sebanyak **264 pasien dirawat ulang**, sekitar dua puluh enam koma delapan tiga persen. Rata-rata biayanya **delapan ribu tiga**

ratus enam puluh tujuh koma empat delapan, dan rata-rata lama rawatnya **tiga puluh tujuh koma enam enam hari**. Di fungsi `tampilkan_ringkasan`, perulangan for membaca setiap rekam, `isinstance` menghitung pasien rawat ulang, total biaya dan lama rawat dibagi jumlah pasien, dan jumlah tiap kondisi dihitung dengan dictionary. *(tunjuk: kotak bawah)* Kondisi yang paling sering muncul ternyata ada dua. **KLIK →**

Kode terkait (dibahas, tidak tampil di slide) · main.py baris 100–111

Baris	Kode	Fungsinya
100	`for rekam in daftar_rekam:`	Mengulang setiap rekam medis di list.
101	`# [INHERITANCE] isinstance() mengenali object dari class PasienRawatUlang`	Komentar: isinstance mengenali object PasienRawatUlang.
102	`if isinstance(rekam.pasien, PasienRawatUlang):`	`isinstance` memeriksa apakah pasien adalah object `PasienRawatUlang` …
103	`jumlah_rawat_ulang += 1`	… jika ya, jumlah pasien rawat ulang ditambah 1.
104	`# [ENKAPSULASI] Data dibaca lewat property, bukan atribut private langsung`	Komentar: data dibaca lewat property.
105	`total_biaya += rekam.biaya`	Menjumlahkan biaya semua pasien.
106	`total_lama_rawat += rekam.lama_rawat`	Menjumlahkan lama rawat semua pasien.
107	`jumlah_per_kondisi[rekam.kondisi] = jumlah_per_kondisi.get(rekam.kondisi, 0) + 1`	Menghitung jumlah pasien per kondisi memakai dictionary; `get(kondisi, 0)` memberi 0 jika kondisi belum ada.
109	`rata_rata_biaya = total_biaya / total_pasien`	Rata-rata biaya = total biaya dibagi jumlah pasien.
110	`rata_rata_lama_rawat = total_lama_rawat / total_pasien`	Rata-rata lama rawat = total lama rawat dibagi jumlah pasien.
111	`persen_rawat_ulang = jumlah_rawat_ulang / total_pasien * 100`	Persentase rawat ulang = jumlah rawat ulang dibagi total pasien, dikali 100.

SLIDE 34 Fractured Leg dan Heart Attack Seri di Posisi Teratas

± 30 detik

(tunjuk: dua batang teratas) Fractured Leg dan Heart Attack sama-sama dialami **67 pasien**, jadi hasilnya **seri**. Sebaran kelima belas kondisi juga hampir merata, antara 64 sampai 67 pasien. Di kode, baris 114 memakai `max` untuk mencari jumlah terbanyak, lalu baris 116 sampai 118 mengumpulkan **semua kondisi** yang jumlahnya sama dengan nilai itu. Karena itu kondisi yang seri tidak hilang; kalau hanya satu yang ditampilkan, hasilnya menyesatkan. **KLIK →**

Kode terkait (dibahas, tidak tampil di slide) · main.py baris 113–123

Baris	Kode	Fungsinya
113	`# Kondisi terbanyak bisa lebih dari satu jika jumlahnya sama (seri)`	Komentar: kondisi terbanyak bisa lebih dari satu (seri).
114	`jumlah_terbanyak = max(jumlah_per_kondisi.values())`	`max` mencari jumlah pasien terbanyak dari semua kondisi.
115	`kondisi_terbanyak = []`	Menyiapkan list kosong untuk kondisi teratas.
116	`for kondisi, jumlah in jumlah_per_kondisi.items():`	Mengulang setiap pasangan kondisi dan jumlahnya.
117	`if jumlah == jumlah_terbanyak:`	Jika jumlahnya sama dengan jumlah terbanyak …
118	`kondisi_terbanyak.append(kondisi)`	… kondisi itu dimasukkan ke list, sehingga kondisi yang seri tidak hilang.
120	`if len(kondisi_terbanyak) > 1:`	Jika kondisi teratas lebih dari satu …
121	`keterangan = f"masing-masing {jumlah_terbanyak} pasien"`	… keterangannya “masing-masing 67 pasien”.

Baris	Kode	Fungsinya
122	`else:`	Jika hanya satu …
123	`keterangan = f"{jumlah_terbanyak} pasien"`	… keterangannya cukup “67 pasien”.

SLIDE 35 Demo Menu: Data Bisa Dilihat, Dicari, dan Ditambah

± 50 detik

Sekarang demo menunya, dengan output asli program. *(tunjuk: terminal kiri atas)* Pilihan 1 menampilkan **sepuluh data pertama** dalam tabel singkat. *(tunjuk: terminal kanan atas)* Pilihan 2 mencari Patient_ID 2: fungsi cari_rekam memeriksa setiap rekam, dan pasien ditemukan sebagai **PasienRawatUlang**, lengkap dengan baris Status Rawat. ID 60, yang memang tidak ada di data, dijawab tidak ditemukan, dan input huruf ditolak oleh try-except. *(tunjuk: terminal kiri bawah)* Pilihan 3 menambah pasien. Baris 205 membuat ID otomatis, yaitu ID terbesar ditambah satu, jadi **1001**, lalu baris 212 memanggil buat_rekam_medis, sehingga data baru diperiksa **setter yang sama** dengan data CSV. *(tunjuk: terminal kanan bawah)* Buktinya, kepuasan 9 ditolak. Pasien yang valid disimpan ke list di baris 217, sedangkan file CSV asli tidak diubah. KLIK →

Kode terkait (dibahas, tidak tampil di slide) · main.py baris 163–174

Baris	Kode	Fungsinya
163	`def menu_lihat_data(daftar_rekam):`	Fungsi `menu_lihat_data()` untuk menu 1.
164	`"""Menu 1: menampilkan 10 data pertama dalam bentuk tabel singkat."""`	Docstring: keterangan fungsi.
165	`cetak_judul(f"{JUMLAH_DATA_MENU} DATA PERTAMA (dari {len(daftar_rekam)} data)")`	Mencetak judul beserta jumlah seluruh data.
166	`print(f"{'ID':<5}{'Umur':<6}{'Gender':<8}{'Kondisi':<23}{'Biaya':>10}  Rawat Ulang")`	Mencetak judul kolom tabel; `:<5` artinya rata kiri selebar 5 karakter, `:>10` rata kanan selebar 10.
167	`for rekam in daftar_rekam[:JUMLAH_DATA_MENU]:`	Mengulang 10 rekam medis pertama (slicing `[:JUMLAH_DATA_MENU]`).
168	`pasien = rekam.pasien`	Mengambil object pasien dari rekam medis (komposisi).
169	`if isinstance(pasien, PasienRawatUlang):`	`isinstance` memeriksa apakah pasien ini object `PasienRawatUlang` …
170	`rawat_ulang = "Ya"`	… jika ya, kolom Rawat Ulang diisi “Ya”.
171	`else:`	Jika bukan …
172	`rawat_ulang = "Tidak"`	… kolom Rawat Ulang diisi “Tidak”.
173	`print(f"{pasien.patient_id:<5}{pasien.umur:<6}{pasien.jenis_kelamin:<8}"`	Mencetak satu baris tabel: ID, umur, dan jenis kelamin dibaca lewat property …
174	`f"{rekam.kondisi:<23}{rekam.biaya:>10,.2f}  {rawat_ulang}")`	… lanjutan baris yang sama: kondisi, biaya (pemisah ribuan, 2 desimal), dan status rawat ulang.

Kode terkait (dibahas, tidak tampil di slide) · main.py baris 177–198

Baris	Kode	Fungsinya
177	`def cari_rekam(daftar_rekam, patient_id):`	Fungsi `cari_rekam()`: mencari rekam medis berdasarkan Patient_ID.
178	`"""Mengembalikan rekam medis dengan Patient_ID tersebut, atau None jika tidak ada."""`	Docstring: mengembalikan rekam medis, atau None jika tidak ada.

Baris	Kode	Fungsinya
179	`for rekam in daftar_rekam:`	Mengulang setiap rekam medis di list.
180	`if rekam.pasien.patient_id == patient_id:`	Jika Patient_ID pasien di rekam ini sama dengan yang dicari …
181	`return rekam`	… rekam itu langsung dikembalikan, dan pencarian berhenti.
182	`return None`	Jika sampai akhir tidak ada yang cocok, mengembalikan None.
185	`def menu_cari_pasien(daftar_rekam):`	Fungsi `menu_cari_pasien()` untuk menu 2.
186	`"""Menu 2: mencari pasien berdasarkan Patient_ID."""`	Docstring: keterangan fungsi.
187	`cetak_judul("CARI PASIEN BERDASARKAN PATIENT_ID")`	Mencetak judul menu.
188	`try:`	`try`: ketikan pengguna akan diubah menjadi angka.
189	`patient_id = int(input("Masukkan Patient_ID: "))`	`input()` membaca ketikan pengguna, lalu `int()` mengubahnya menjadi bilangan bulat.
190	`except ValueError:`	`except ValueError`: menangkap error jika yang diketik bukan angka, misalnya “abc”.
191	`print("Patient_ID harus berupa angka.")`	Mencetak pesan bahwa Patient_ID harus berupa angka.
192	`return`	`return` kembali ke menu tanpa mencari.
193	`rekam = cari_rekam(daftar_rekam, patient_id)`	Memanggil `cari_rekam()` dengan Patient_ID yang diketik.
194	`if rekam is None:`	Jika hasilnya None …
195	`print(f"Pasien dengan Patient_ID {patient_id} tidak ditemukan.")`	… mencetak pesan pasien tidak ditemukan.
196	`else:`	Jika ditemukan …
197	`print(f"Pasien ditemukan (object dari class: {type(rekam.pasien).__name__})")`	… mencetak nama class object pasiennya (`Pasien` atau `PasienRawatUlang`) …
198	`rekam.tampilkan_rekam()`	… lalu menampilkan rekam medis lengkap lewat `tampilkan_rekam()`.

Kode terkait (dibahas, tidak tampil di slide) · main.py baris 201–218

Baris	Kode	Fungsinya
201	`def menu_tambah_pasien(daftar_rekam):`	Fungsi `menu_tambah_pasien()` untuk menu 3.
202	`"""Menu 3: menambah pasien baru ke list (file CSV asli tidak diubah)."""`	Docstring: pasien baru disimpan ke list; file CSV asli tidak diubah.
203	`cetak_judul("TAMBAH PASIEN BARU")`	Mencetak judul menu.
204	`# Patient_ID dibuat otomatis (ID terbesar + 1) agar tidak ada ID ganda`	Komentar: cara membuat Patient_ID baru.
205	`id_baru = max(rekam.pasien.patient_id for rekam in daftar_rekam) + 1`	`max()` mencari Patient_ID terbesar dari semua rekam, lalu ditambah 1 (data asli: 1000 → 1001), sehingga tidak ada ID ganda.
206	`print(f"Patient_ID baru (otomatis) : {id_baru}")`	Menampilkan Patient_ID baru kepada pengguna.
207	`baris = {"Patient_ID": str(id_baru)}`	Menyiapkan dictionary `baris` berbentuk sama seperti satu baris CSV, diawali Patient_ID.
208	`for kolom, pertanyaan in PERTANYAAN_PASIEN_BARU:`	Mengulang setiap pasangan (kolom, pertanyaan) dari `PERTANYAAN_PASIEN_BARU`.
209	`baris[kolom] = input(f"{pertanyaan:<28}: ").strip()`	Menampilkan pertanyaan, membaca jawaban dengan `input()`, membuang spasi dengan `strip()`, lalu menyimpannya di kolom yang sesuai.
210	`try:`	`try`: mencoba membuat object dari jawaban pengguna.
211	`# [ENKAPSULASI] Input diperiksa oleh setter yang sama dengan data CSV`	Komentar: input diperiksa oleh setter yang sama dengan data CSV (enkapsulasi).
212	`rekam_baru = buat_rekam_medis(baris)`	Memanggil `buat_rekam_medis()`, fungsi yang juga dipakai untuk setiap baris CSV, sehingga semua validasi setter berlaku.

Baris	Kode	Fungsinya
213	`except ValueError as error:`	`except ValueError`: menangkap data yang tidak valid, misalnya kepuasan 9.
214	`print(f"Data tidak valid: {error}")`	Mencetak alasan data ditolak.
215	`print("Pasien baru tidak ditambahkan.")`	Mencetak bahwa pasien tidak ditambahkan.
216	`return`	`return` kembali ke menu tanpa menambah data.
217	`daftar_rekam.append(rekam_baru)`	Jika valid, object rekam medis baru ditambahkan ke list dengan append.
218	`print(f"Pasien ID {id_baru} berhasil ditambahkan. Jumlah data sekarang: {len(daftar_rekam)}")`	Mencetak pesan berhasil beserta jumlah data sekarang.

SLIDE 36 Ketiga Konsep OOP Berhasil Diterapkan pada Data Nyata

± 30 detik

Sebagai kesimpulan, ketiga konsep OOP berhasil kami terapkan pada data nyata. *(tunjuk: kolom 1)* **Class dan object**: empat class di file terpisah, dan 984 baris CSV menjadi 1968 object. *(tunjuk: kolom 2)* **Enkapsulasi**: semua atribut private dan divalidasi setter, termasuk data dari menu. *(tunjuk: kolom 3)* **Inheritance**: Orang ke Pasien ke PasienRawatUlang, dengan super, overriding, dan method khusus status_rawat. *(tunjuk: kotak bawah)* Ke depan, program bisa dikembangkan dengan fitur ubah dan hapus data, atau penyimpanan ke database. **KLIK →**

SLIDE 37 Terima Kasih

± 10 detik

Sekian presentasi dari **Kelompok 3**. Terima kasih atas perhatian Bapak/Ibu dosen dan teman-teman. Kami persilakan jika ada pertanyaan. *(jeda)* *(tunjuk: tetap di slide ini selama tanya jawab)*

Kalimat penutup bagianmu: “Sekian presentasi dari Kelompok 3.”

Persiapan tanya jawab

T **Kenapa kondisi paling sering ada dua?**

J Karena Fractured Leg dan Heart Attack sama-sama muncul 67 kali (seri). Program sengaja menampilkan semua kondisi dengan jumlah tertinggi agar hasilnya jujur.

T **Kenapa rata-rata lama rawat cukup tinggi (37.66 hari)?**

J Itu hasil dari data asli. Lama rawat di dataset berkisar 1 sampai 76 hari, dan program hanya menghitung rata-ratanya.

T **Kenapa hanya 5 rekam medis yang ditampilkan?**

J Supaya output tetap ringkas dan mudah dibaca. Seluruh 984 data tetap dimuat dan dipakai untuk menghitung ringkasan.

T **Apakah pasien yang ditambah lewat menu tersimpan ke file CSV?**

J Tidak. Pasien baru disimpan di list selama program berjalan, jadi file CSV asli tetap utuh. Penyimpanan permanen bisa menjadi pengembangan berikutnya.

T **Kenapa ID pasien baru 1001?**

J ID dibuat otomatis dari ID terbesar ditambah satu. ID terbesar di data adalah 1000, jadi pasien baru mendapat 1001 dan tidak ada ID ganda.

T **Bagaimana jika diminta demo langsung?**

J Jalankan python main.py, lalu ketik masukan yang sama dengan slide demo: 1; 2 lalu 2; 2 lalu 60; 3 lalu 34, Female, Fractured Arm, X-Ray and Splint, 500, 2, No, Recovered, 5; 2 lalu 1001; 4. Hasilnya akan sama dengan output di slide.

SESI TANYA JAWAB

Pertanyaan dijawab oleh anggota yang membawakan topiknya. Alfira (moderator penutup) mengulang pertanyaan, lalu mempersilakan anggota yang sesuai.

Topik pertanyaan	Dijawab oleh	Slide
Latar belakang, tujuan, struktur file, cara menjalankan	**Riswan**	1–5
Dataset, kolom, dan aturan nilai	**Yus Aidil**	6–8
Class, object, constructor, self	**Fahmi**	9–12
Atribut private, property, setter, ValueError	**Diva Nurislamia Agfa**	13–16
Pewarisan, super(), method overriding	**Rosa Ramadani**	17–19
Pewarisan bertingkat, polimorfisme, MRO	**Sofiang**	20–22
Komposisi, isinstance, biaya per hari	**Revalinda**	23–26
main.py, membaca CSV, try-except	**Andi Rezky Nabila**	27–30
Output, ringkasan, kondisi seri, kesimpulan	**Alfira Yunita Syam**	31–37

Jika belum yakin dengan jawabannya

- Ulangi pertanyaannya dengan kalimat sendiri untuk memastikan maksudnya.
- Tunjukkan buktinya di slide: klik kartu anggota di slide **Pembagian Materi** untuk melompat ke bagian terkait, lalu tunjuk baris kodenya.
- Kalimat aman: “Terima kasih atas pertanyaannya. Izin kami tunjukkan bagian kodenya agar lebih jelas.”
- Bank tanya jawab lengkap ada di slide cadangan tersembunyi dan di setiap bagian naskah ini.